实验三 聚类分析

一、 实验目的

通过实验更好地掌握机器学习及优化相关概念、技术、原理、应用等，提高编写实验报告、总结实验结果的能力；对机器学习模型、优化算法等有比较深入的认识。需掌握的知识点如下：

- 掌握机器学习中涉及的相关概念、模型、算法；
- 熟悉机器学习模型训练、验证、测试的流程；
- 熟悉常用的数据预处理方法；
- 掌握聚类分析问题的表示、求解及编程。

二、基本要求

准备好实验数据，编程完成实验内容，收集实验结果。

独立完成实验报告。

三、实验软件

推荐使用 Python 编程语言（允许使用 numpy 库，需实现详细实验步骤，不允许直接调用 scikit-learn 中回归、分类、聚类等高层 API）。

四、实验内容

基于 IRIS 鸢尾花数据集，完成关于鸢尾花的聚类分析。

（1）准备数据集并认识数据

下载 IRIS 数据集

https://archive.ics.uci.edu/ml/datasets/iris

列名	说明	类型
SepalLength	花萼长度	float
SepalWidth	花萼宽度	float
PetalLength	花瓣长度	float
PetalWidth	花瓣宽度	float
Class	类别变量：0 表示山鸢尾，1 表示变色鸢尾，2 表示维吉尼亚鸢尾	int

了解数据集各个维度特征的含义

（2）探索数据并预处理数据

观察数据集各个维度特征的数值类型与分布

挑选 sepal length、petal length 两维特征作为聚类依据

（3）求解聚类中心

编程实现 k-means 聚类、混合高斯聚类

（4）测试和评估模型

在数据集上计算聚类的性能指标

五、学生实验报告

（1）简要介绍 k-means、混合高斯聚类的原理

k-means 原理：

k-means 算法是常用的一种聚类算法。算法的输入为一个样本集（点集），通过该算法可以将样本进行聚类，具有相似特征的样本聚为一类。

算法思想：

假设要把数据分成K个类，则可分为以下步骤：

① 随机选K个点，作为聚类中心

② 计算每个点分别到K个聚类中心的距离，然后将该点分给距离最近的那个中心点，这样就形成了K个簇

③ 再重新计算每个簇的质心（均值）

④ 重复 2-3 步，直到质心的位置不在发生变化或者达到设定的迭代次数

混合高斯聚类原理：

①多元高斯分布的概率密度函数为：

$$p(\boldsymbol{x})=\frac{1}{(2\pi)^{n/2}|\Sigma|^{-1}}e^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{T}\Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})}$$

其中$\boldsymbol{x}$是单个样本（d维向量），$\boldsymbol{\mu}$是分布均值（d维向量）；$\boldsymbol{\Sigma}$分布协方差矩阵（$d\times d$矩阵，对称半正定矩阵）；$\boldsymbol{\Sigma}^{-1}$协方差矩阵的逆，$|\boldsymbol{\Sigma}|$是协方差矩阵的行列式。返回值是一个标量，对应样本$\boldsymbol{x}$的概率密度值。

可以看出，概率密度值由后面两个变量决定，一个是均值$\boldsymbol{\mu}$，一个是协方差矩阵$\boldsymbol{\Sigma}$。

高斯混合聚类是从若干个满足高斯分布的集合中抽取而成。

观测数据$\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_N$由高斯混合模型生成，

$$P_{\boldsymbol{\theta}}(\boldsymbol{x})=\sum_{k=1}^{K}\alpha_k\, p(\boldsymbol{x}|\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k)$$

令$\boldsymbol{\theta}=(\alpha_1, \alpha_2, \cdots, \alpha_K; \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \cdots, \boldsymbol{\mu}_K; \boldsymbol{\Sigma}_1, \boldsymbol{\Sigma}_2, \cdots, \boldsymbol{\Sigma}_K)$

我们用 EM 算法估计高斯混合模型的数$\boldsymbol{\theta}$。

②初始化模型参数：$(\hat{\boldsymbol{\mu}}_k, \hat{\boldsymbol{\Sigma}}_k, \hat{\alpha}_k), k=1, 2, \cdots, K$

③EM 算法的 E 步：

计算

$$\hat{\gamma}_{jk}=\frac{\alpha_k\emptyset(\boldsymbol{x}_j|\boldsymbol{\theta}_k)}{\sum_{k=1}^{K}\alpha_k\emptyset(\boldsymbol{x}_j|\boldsymbol{\theta}_k)}, j=1, 2, \cdots, N, k=1, 2, \cdots, K$$

（有点相当于朴素贝叶斯中计算后验概率，用先验概率乘以条件概率）

这是当前模型参数下第j个观测数据来自第k个分模型的概率，称为分模型k对观测数据y_j的影响度。

④EM 算法的 M 步：更新模型参数

$$\hat{\boldsymbol{\mu}}_k=\frac{\sum_{j=1}^{N}\hat{\gamma}_{jk}\boldsymbol{x}_j}{\sum_{j=1}^{N}\hat{\gamma}_{jk}}$$

$$\hat{\boldsymbol{\Sigma}}_k=\frac{\sum_{j=1}^{N}\hat{\gamma}_{jk}(\boldsymbol{x}_j-\hat{\boldsymbol{\mu}}_k)(\boldsymbol{x}_j-\hat{\boldsymbol{\mu}}_k)^{T}}{\sum_{j=1}^{N}\hat{\gamma}_{jk}} \quad 或\hat{\boldsymbol{\Sigma}}_k=\frac{(\boldsymbol{X}-\hat{\boldsymbol{\mu}}_k)^{T}((\boldsymbol{X}-\hat{\boldsymbol{\mu}}_k)\odot\boldsymbol{\gamma}_i)}{\sum_{j=1}^{N}\hat{\gamma}_{jk}}$$

$$\hat{\alpha}_k=\frac{\sum_{j=1}^{N}\hat{\gamma}_{jk}}{N}$$

其中，$k=1, 2, \cdots, K$；$\boldsymbol{\gamma}_i$是第i个组件的权重列向量，$\odot$是按元素乘法。

重复 E 步和 M 步直到模型收敛。

（2）程序清单（包含详细求解步骤）

k-means 聚类算法

①导入相应库

```
import numpy as np
import pandas as pd
import random
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
```

②加载数据集，观察数据特点

```
iris_data=pd.read_csv( filepath_or_buffer: "Iris/iris.data",header=None,names=['sepal length','sepal width','petal length',
                                                                                'petal width','class'])
print(iris_data.info())
#发现有iris-setosa   iris-versicolor   iris-virginica三种
print(iris_data['class'].value_counts())
```

③挑选 sepal length、petal length 两维特征作为聚类依据，将类别“class”列的值赋给 labels，并进行标签编码

```
#对labels进行标签编码
labels=iris_data['class'].values
label_encoder=LabelEncoder()
labels=label_encoder.fit_transform(labels)
# print(labels)

#挑选sepal length、petal length两维特征作为聚类依据
x_axis=iris_data['sepal length']  #series(150,)
y_axis=iris_data['petal length']

print(x_axis.shape)
print(y_axis.shape)
```

④初始化聚类中心

```
#随机三个index值，在150条数据集中随机选中三个种类的开始点的标号
indexList=random.sample(range(0,150), k: 3)
print(indexList)

#随机初始的中心点
x_center1=x_axis[indexList[0]]
y_center1=y_axis[indexList[0]]
x_center2=x_axis[indexList[1]]
y_center2=y_axis[indexList[1]]
x_center3=x_axis[indexList[2]]
y_center3=y_axis[indexList[2]]
```

⑤开始训练：计算每个点分别到k个聚类中心的距离，然后将该点分给距离最近的那个中心点

```python
for j in range(150):
    belong=0          #belong用来记录该条数据属于的类别
    dis_1=pow((x_axis[j]-x_center1),2)+pow((y_axis[j]-y_center1),2)
    dis_2=pow((x_axis[j]-x_center2),2)+pow((y_axis[j]-y_center2),2)
    dis_3=pow((x_axis[j]-x_center3),2)+pow((y_axis[j]-y_center3),2)

    #比较离三类中心点哪个更近，将该条数据归于距离更近的中心点所属类
    if dis_2<dis_1:
        belong=2
        if dis_3<dis_2:
            belong=3
    else:
        belong=1
        if dis_3<dis_1:
            belong=3
    # print(belong)
    if belong==1:
        belong1.append(j)
    elif belong==2:
        belong2.append(j)
    else:
        belong3.append(j)
```

⑥重新计算每个簇的质心（均值）

```python
#进行center点的位置更新
for k in range(len(belong1)):
    x_center1+=x_axis[belong1[k]]
    y_center1+=y_axis[belong1[k]]
for k in range(len(belong2)):
    x_center2 += x_axis[belong2[k]]
    y_center2 += y_axis[belong2[k]]
for k in range(len(belong3)):
    x_center3 += x_axis[belong3[k]]
    y_center3 += y_axis[belong3[k]]

x_center1=x_center1/(1+len(belong1))
x_center2=x_center2/(1+len(belong2))
x_center3=x_center3/(1+len(belong3))
y_center1 = y_center1 / (1 + len(belong1))
y_center2 = y_center2 / (1 + len(belong2))
y_center3 = y_center3 / (1 + len(belong3))
```

⑦迭代第⑤、⑥步 max_iter=1000 次

⑧显示 k-means 算法的聚类情况，同时也显示实际数据的归类情况

```python
y_pred=np.array(np.zeros(150))
for i in range(len(belong1)):
    y_pred[belong1[i]]=1

for i in range(len(belong2)):
    y_pred[belong2[i]]=2

for i in range(len(belong3)):
    y_pred[belong3[i]]=3

#k-means求出的聚类中心
x_center=[x_center1,x_center2,x_center3]
y_center=[y_center1,y_center2,y_center3]
#数据集的实际中心
x_ac_center=[x_axis[0:50].mean(),x_axis[50:100].mean(),x_axis[100:150].mean()]
y_ac_center=[y_axis[0:50].mean(),y_axis[50:100].mean(),y_axis[100:150].mean()]
#画图
#聚类算法的归类情况
plt.scatter(x_axis,y_axis,c=y_pred)
plt.scatter(x_center,y_center,c='r',marker='x')
plt.show()

#实际数据的归类情况
plt.scatter(x_axis,y_axis,c=labels)
plt.scatter(x_ac_center,y_ac_center,c='r',marker='x')
plt.show()
```

⑨计算准确率（由于聚类算法只会将原始数据样本划分为K个簇，但是并不会告诉我们每个簇分别对应哪个类别，因此用排列组合的方法，分别计算每种情况的准确率，选最高的为最终的准确率值）。

```python
y_pred_1=np.array(np.zeros(150))
y_pred_2=np.array(np.zeros(150))
y_pred_3=np.array(np.zeros(150))
y_pred_4=np.array(np.zeros(150))
y_pred_5=np.array(np.zeros(150))
y_pred_6=np.array(np.zeros(150))
for i in range(150):
    if y_pred[i]==1:
        y_pred_1[i]=0
        y_pred_2[i] = 1
        y_pred_3[i] = 0
        y_pred_4[i] = 1
        y_pred_5[i] = 2
        y_pred_6[i] = 2
    if y_pred[i]==2:
        y_pred_1[i] = 1
        y_pred_2[i] = 0
        y_pred_3[i] = 2
        y_pred_4[i] = 2
        y_pred_5[i] = 1
        y_pred_6[i] = 0
    if y_pred[i]==3:
        y_pred_1[i] = 2
        y_pred_2[i] = 2
        y_pred_3[i] = 1
        y_pred_4[i] = 0
        y_pred_5[i] = 0
        y_pred_6[i] = 1
```

```python
def correct_rate(lei_list):  6 用法
    correct_num = 0
    for i in range(150):
        if (lei_list[i] == labels[i]):
            correct_num += 1
    rate = correct_num / 150
    return rate

rate1=correct_rate(y_pred_1)
rate2=correct_rate(y_pred_2)
rate3=correct_rate(y_pred_3)
rate4=correct_rate(y_pred_4)
rate5=correct_rate(y_pred_5)
rate6=correct_rate(y_pred_6)

#比较
rate=[rate1,rate2,rate3,rate4,rate5,rate6]
max_rate=0
for i in range(6):
    if rate[i]>max_rate:
        max_rate=rate[i]

print('准确率为: ',max_rate)
```

解释：可以用表格表示

y_pred	y_pred_1	y_pred_2	y_pred_3	y_pred_4	y_pred_5	y_pred_6
1	0	1	0	1	2	2
2	1	0	2	2	1	0
3	2	2	1	0	0	1

混合高斯算法

①引入相应库

```python
from scipy.stats import multivariate_normal
from sklearn import preprocessing
from sklearn.metrics import accuracy_score
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
```

②导入数据集，观察数据特点

```python
iris_data=pd.read_csv( filepath_or_buffer: "iris.data",header=None,names=['sepal length','sepal width','petal length',
                                                                          'petal width','class'])

#发现有iris-setosa   iris-versicolor   iris-virginica三种
print(iris_data['class'].value_counts())
labels=iris_data['class'].values
```

③挑选 sepal length、petal length 两维特征作为聚类依据，用 data 存储这两列数据。labels 存储类别‘class’列的值,并进行标签编码

```python
#对labels进行标签编码
label_encoder=LabelEncoder()
labels=label_encoder.fit_transform(labels)
# print(labels)

#挑选sepal length、petal length两维特征作为聚类依据
x_axis=iris_data['sepal length']  #series(150,)
y_axis=iris_data['petal length']

data=np.array(pd.concat( objs: [x_axis,y_axis],axis=1))
```

④实例化类 GMM_EM 的对象 gmm

```python
gmm = GMM_EM(3)
```

执行类的__init__函数，确定进行聚类的类数为 3 即 n_components=3

```python
class GMM_EM():  1个用法
    def __init__(self, n_components, max_iter=1000, error=1e-6):
        self.n_components = n_components  # 混合模型由几个gauss模型组成
        self.max_iter = max_iter  # 最大迭代次数
        self.error = error  # 收敛误差
        self.samples = 0   #样本个数
        self.features = 0 #存储特征个数
        self.alpha = []  # 存储模型权重
        self.mu = []  # 存储均值
        self.sigma = []  # 存储标准差
```

⑤调用对象 gmm 中的 fit_predict 函数，得到用混合高斯模型进行聚类的结果

```python
pre_label = gmm.fit_predict(data)
```

```python
def fit_predict(self, data):  # 拟合数据  1个用法
    data = self.preprocess(data)   #进行数据预处理
    self._init(data)     #初始化模型参数
    weighted_probs = np.zeros((self.samples, self.n_components))
    print(weighted_probs.shape)  #用来存放进行E步后算出来的当前模型参数下每个观测数据来自第k个分模型的概率 shape(150,3)
    for i in range(self.max_iter):
        prev_weighted_probs = weighted_probs
        #e步
        weighted_probs = self._e_step(data)
        #change 当后验概率没有什么改变了的时候，即收敛的时候，停止迭代
        change = np.linalg.norm(weighted_probs - prev_weighted_probs)
        if change < self.error:
            break
        #m步
        self._m_step(data, weighted_probs)
    #比较每个观测数据来自三个分模型的概率，返回概率最大的分模型所代表列的列号
    return weighted_probs.argmax(axis=1)
```

分析 fit_predict(data)函数内进行的步骤：

1）进行数据预处理，调用类内函数 preprocess()

```python
def preprocess(self, data):  # 数据预处理  1个用法
    self.samples = data.shape[0]   #定义数据集大小
    self.features = data.shape[1]   #定义数据集的特征数
    pre = preprocessing.MinMaxScaler()  #进行了特征归一化
    return pre.fit_transform(data)
```

data 数据集中大小为 150，特征数为 2

2）调用类内函数_init()初始化高斯模型的参数

```python
def _init(self, data):  # 初始化参数  1个用法
    np.random.seed(4)
    self.mu = np.array(np.random.rand(self.n_components, self.features))
    #sigma中为每个分模型都初始化了一个二维随机变量的协方差矩阵
    self.sigma = np.array([np.eye(self.features) / self.features] * self.n_components)
    self.alpha = np.array([1.0 / self.n_components] * self.n_components)
    print(self.alpha.shape, self.mu.shape, self.sigma.shape)
    print(self.alpha,self.mu,self.sigma)
```

3）在 max_iter 迭代次数为 1000 的情况下执行 EM 算法中的 E、M 步骤，当前后两次迭代概率的变化＜1e-6 时，即可跳出迭代。

E 步骤的函数对应该算法

$$\hat{\gamma}_{jk}=\frac{\alpha_k \emptyset(y_j|\theta_k)}{\sum_{k=1}^{K}\alpha_k \emptyset(y_j|\theta_k)}, j=1, 2, \cdots, N, k=1, 2, \cdots, K$$

```python
def _e_step(self, data):  # E步  1个用法
    probs = np.zeros((self.samples, self.n_components))   #shape(150,3)
    for i in range(self.n_components):
        #调用类定义的gauss函数，计算不同高斯模型下数据集对应高斯函数值
        probs[:, i] = self.gauss(data, self.mu[i, :], self.sigma[i, :, :])

    weighted_probs = np.zeros(probs.shape)
    for i in range(self.n_components):
        weighted_probs[:, i] = self.alpha[i] * probs[:, i]
    for i in range(self.samples):
        #后验概率某类概率再除以三类的概率和
        weighted_probs[i, :] /= np.sum(weighted_probs[i, :])

    return weighted_probs
```

_e_step()函数内调用的 guass()函数定义如下：

```python
def gauss(self, Y, mu, sigma):  # 直接调用多元正态分布的概率密度函数，计算高斯函数值  1个用法
    return multivariate_normal.pdf(Y,mean=mu, cov=sigma )
```

M 步骤的函数对应算法：

$$\hat{\boldsymbol{\mu}}_k=\frac{\sum_{j=1}^{N}\hat{\gamma}_{jk}\boldsymbol{x}_j}{\sum_{j=1}^{N}\hat{\gamma}_{jk}}$$

$$\hat{\boldsymbol{\Sigma}}_k=\frac{\sum_{j=1}^{N}\hat{\gamma}_{jk}(\boldsymbol{x}_j-\hat{\boldsymbol{\mu}}_k)(\boldsymbol{x}_j-\hat{\boldsymbol{\mu}}_k)^{\mathrm{T}}}{\sum_{j=1}^{N}\hat{\gamma}_{jk}} \quad 或\hat{\boldsymbol{\Sigma}}_k=\frac{(\boldsymbol{X}-\hat{\boldsymbol{\mu}}_k)^{\mathrm{T}}((\boldsymbol{X}-\hat{\boldsymbol{\mu}}_k)\odot\boldsymbol{\gamma}_i)}{\sum_{j=1}^{N}\hat{\gamma}_{jk}}$$

$$\hat{\alpha}_k=\frac{\sum_{j=1}^{N}\hat{\gamma}_{jk}}{N}$$

```python
def _m_step(self, data, weighted_probs):  # M步  进行mu,sigma,alpha的数值更新  1个用法
    for i in range(self.n_components):
        #求出每一列的概率和 即每行数据属于某一具体类的概率和
        sum_probs_i = np.sum(weighted_probs[:, i])
        #axis=0计算每列的sum
        self.mu[i, :] = np.sum(np.multiply(data, np.asmatrix(weighted_probs[:, i]).T), axis=0) / sum_probs_i

        self.sigma[i, :, :] = (data - self.mu[i, :]).T * np.multiply((data - self.mu[i, :]),
                                                                     np.asmatrix(weighted_probs[:, i]).T) / sum_probs_i

        #行数  shape[0]
        self.alpha[i] = sum_probs_i / data.shape[0]
```

⑥求解混合高斯模型聚类中心

```python
num_0,num_1,num_2=[0,0,0]
xsum_0,xsum_1,xsum_2=[0,0,0]
ysum_0,ysum_1,ysum_2=[0,0,0]

for i in range(len(pre_label)):
    if pre_label[i]==0:
        num_0+=1
        xsum_0+=x_axis[i]
        ysum_0 += y_axis[i]
    elif pre_label[i]==1:
        num_1+=1
        xsum_1+=x_axis[i]
        ysum_1 += y_axis[i]
    else:
        num_2+=1
        xsum_2+=x_axis[i]
        ysum_2 += y_axis[i]

x_center_0=xsum_0/num_0
y_center_0=ysum_0/num_0
x_center_1=xsum_1/num_1
y_center_1=ysum_1/num_1
x_center_2=xsum_2/num_2
y_center_2=ysum_2/num_2

x_center=[x_center_0,x_center_1,x_center_2]
y_center=[y_center_0,y_center_1,y_center_2]
#数据集的实际中心
x_ac_center=[x_axis[0:50].mean(),x_axis[50:100].mean(),x_axis[100:150].mean()]
y_ac_center=[y_axis[0:50].mean(),y_axis[50:100].mean(),y_axis[100:150].mean()]
```

⑦画图显示 k-means 算法的聚类情况及中心点，同时也显示实际数据的归类情况

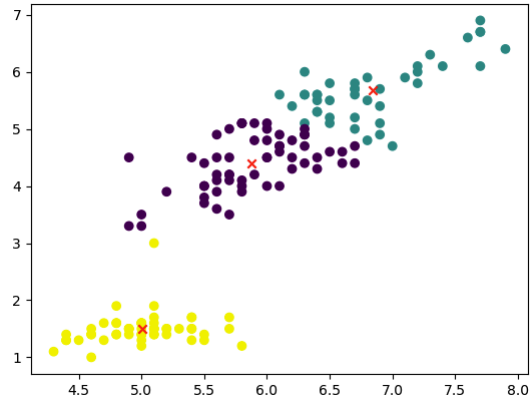
```python
#画图
#画混合高斯聚类图
plt.scatter(x_axis,y_axis,c=pre_label)
plt.scatter(x_center,y_center,c='r',marker='x')
plt.xlabel('sepal length')
plt.ylabel('petal length')
plt.show()

#实际数据图
plt.scatter(x_axis,y_axis,c=labels)
plt.scatter(x_ac_center,y_ac_center,c='r',marker='x')
plt.xlabel('sepal length')
plt.ylabel('petal length')
plt.show()
```

⑧计算准确率，方法与 K-means 中的相同

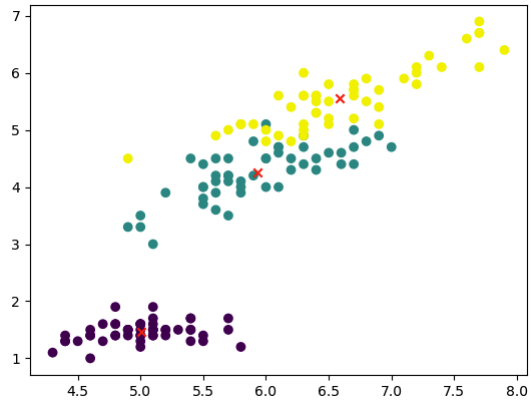
（3）展示实验结果，可视化聚类结果

k-means 算法：

k-means 算法的聚类情况：



数据的实际归类情况：



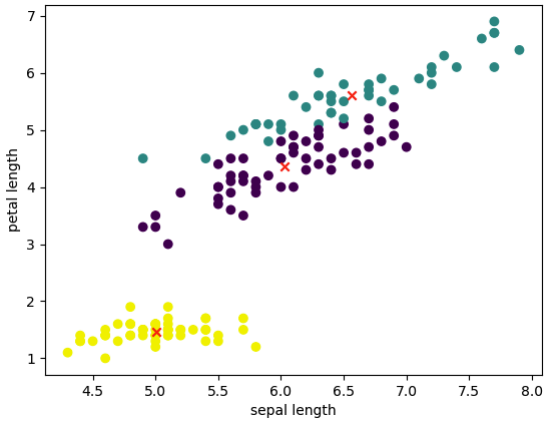
该聚类算法得出准确率为：

```
准确率为:  0.88
```

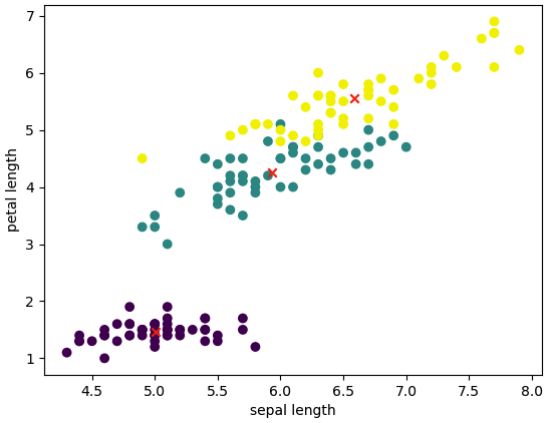
混合高斯算法：（EM 算法是对初始值敏感的，修改初始化的值会发现模型性能变化很大）
固定一个分类较好的随机情况：

```
def _init(self, data):  # 初始化参数 1个用法
    np.random.seed(4)
    self.mu = np.array(np.random.rand(self.n_components, self.features))
    #sigma中为每个分模型都初始化了一个二维随机变量的协方差矩阵
    self.sigma = np.array([np.eye(self.features) / self.features] * self.n_components)
    self.alpha = np.array([1.0 / self.n_components] * self.n_components)
    print(self.alpha.shape, self.mu.shape, self.sigma.shape)
    print(self.alpha,self.mu,self.sigma)
```

此时混合高斯算法聚类情况：



数据实际分类情况：



此时该聚类算法得出准确率为：

```
准确率为:  0.92
```

（5）源代码

#k-means 算法源码

```
import numpy as np
import pandas as pd
import random
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

iris_data=pd.read_csv("Iris/iris.data", header=None, names=['sepal length', 'sepal width', 'petal length', 'petal width', 'class'])

print(iris_data.info())

#发现有 iris-setosa    iris-versicolor    iris-virginica 三种
print(iris_data['class'].value_counts())

#对 labels 进行标签编码
labels=iris_data['class'].values
label_encoder=LabelEncoder()
labels=label_encoder.fit_transform(labels)
# print(labels)

#挑选 sepal length、petal length 两维特征作为聚类依据
x_axis=iris_data['sepal length']   #series(150,)
y_axis=iris_data['petal length']

print(x_axis.shape)
print(y_axis.shape)
```

```
#随机三个 index 值，在 150 条数据集中随机选中三个种类的开始点的标号
indexList=random.sample(range(0,150),3)
print(indexList)

#随机初始的中心点
x_center1=x_axis[indexList[0]]
y_center1=y_axis[indexList[0]]
x_center2=x_axis[indexList[1]]
y_center2=y_axis[indexList[1]]
x_center3=x_axis[indexList[2]]
y_center3=y_axis[indexList[2]]

print(x_center1)
print(x_axis[0])

#---------------------开始训练    训练 100 次-------------------------
for i in range(100):
    #  用来装分属于三类的数据的 index 值
    belong1 = []
    belong2 = []
    belong3 = []

    #计算每条分数据别到 3 个聚类中心的距离
    for j in range(150):
        belong=0                #belong 用来记录该条数据属于的类别
        dis_1=pow((x_axis[j]-x_center1),2)+pow((y_axis[j]-y_center1),2)
        dis_2=pow((x_axis[j]-x_center2),2)+pow((y_axis[j]-y_center2),2)
        dis_3=pow((x_axis[j]-x_center3),2)+pow((y_axis[j]-y_center3),2)

        #比较离三类中心点哪个更近，将该条数据归于距离更近的中心点所属类
        if dis_2<dis_1:
            belong=2
            if dis_3<dis_2:
                belong=3
        else:
            belong=1
            if dis_3<dis_1:
                belong=3
        # print(belong)
        if belong==1:
            belong1.append(j)
        elif belong==2:
            belong2.append(j)
```

```
        else:
            belong3.append(j)

    #进行 center 点的位置更新
    for k in range(len(belong1)):
        x_center1+=x_axis[belong1[k]]
        y_center1+=y_axis[belong1[k]]
    for k in range(len(belong2)):
        x_center2 += x_axis[belong2[k]]
        y_center2 += y_axis[belong2[k]]
    for k in range(len(belong3)):
        x_center3 += x_axis[belong3[k]]
        y_center3 += y_axis[belong3[k]]

    x_center1=x_center1/(1+len(belong1))
    x_center2=x_center2/(1+len(belong2))
    x_center3=x_center3/(1+len(belong3))
    y_center1 = y_center1 / (1 + len(belong1))
    y_center2 = y_center2 / (1 + len(belong2))
    y_center3 = y_center3 / (1 + len(belong3))


#y_pred 用来装 k-means 聚类算法对每条数据所归到的类
#注意这里的类值 1、2、3 没有实际意义，
#跟实际数据 labels 中用标签编码所得的值 0、1、2 值没有对应关系
#仅仅是为了区分类别
y_pred=np.array(np.zeros(150))

for i in range(len(belong1)):
    y_pred[belong1[i]]=1

for i in range(len(belong2)):
    y_pred[belong2[i]]=2

for i in range(len(belong3)):
    y_pred[belong3[i]]=3

#k-means 求出的聚类中心
x_center=[x_center1, x_center2, x_center3]
y_center=[y_center1, y_center2, y_center3]
#数据集的实际中心
x_ac_center=[x_axis[0:50].mean(), x_axis[50:100].mean(), x_axis[100:150].mean()]
y_ac_center=[y_axis[0:50].mean(), y_axis[50:100].mean(), y_axis[100:150].mean()]
#画图
```

```
#聚类算法的归类情况
plt.scatter(x_axis,y_axis,c=y_pred)
plt.scatter(x_center,y_center,c='r',marker='x')
plt.show()

#实际数据的归类情况
plt.scatter(x_axis,y_axis,c=labels)
plt.scatter(x_ac_center,y_ac_center,c='r',marker='x')
plt.show()

#计算准确率
#分别计算三种类别组合   0 1 2      1 0 2     0 2 1      1 2 0      2 1 0     2 0 1
y_pred_1=np.array(np.zeros(150))
y_pred_2=np.array(np.zeros(150))
y_pred_3=np.array(np.zeros(150))
y_pred_4=np.array(np.zeros(150))
y_pred_5=np.array(np.zeros(150))
y_pred_6=np.array(np.zeros(150))
for i in range(150):
    if y_pred[i]==1:
        y_pred_1[i]=0
        y_pred_2[i] = 1
        y_pred_3[i] = 0
        y_pred_4[i] = 1
        y_pred_5[i] = 2
        y_pred_6[i] = 2
    if y_pred[i]==2:
        y_pred_1[i] = 1
        y_pred_2[i] = 0
        y_pred_3[i] = 2
        y_pred_4[i] = 2
        y_pred_5[i] = 1
        y_pred_6[i] = 0
    if y_pred[i]==3:
        y_pred_1[i] = 2
        y_pred_2[i] = 2
        y_pred_3[i] = 1
        y_pred_4[i] = 0
        y_pred_5[i] = 0
        y_pred_6[i] = 1

def correct_rate(lei_list):
    correct_num = 0
    for i in range(150):
```

```
        if (lei_list[i] == labels[i]):
            correct_num += 1
    rate = correct_num / 150
    return rate

rate1=correct_rate(y_pred_1)
rate2=correct_rate(y_pred_2)
rate3=correct_rate(y_pred_3)
rate4=correct_rate(y_pred_4)
rate5=correct_rate(y_pred_5)
rate6=correct_rate(y_pred_6)

#比较
rate=[rate1, rate2, rate3, rate4, rate5, rate6]
max_rate=0
for i in range(6):
    if rate[i]>max_rate:
        max_rate=rate[i]

print('准确率为：',max_rate)
```

#混合高斯聚类算法源码

```
from scipy.stats import multivariate_normal
from sklearn import preprocessing
from sklearn.metrics import accuracy_score
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt

class GMM_EM():
    def __init__(self, n_components, max_iter=1000, error=1e-6):
        self.n_components = n_components    #混合模型由几个 gauss 模型组成
        self.max_iter = max_iter    #最大迭代次数
        self.error = error      #收敛误差
        self.samples = 0      #样本个数
        self.features = 0      #存储特征个数
        self.alpha = []         #存储模型权重
        self.mu = []           #存储均值
        self.sigma = []       #存储标准差

    def _init(self, data):    #初始化参数
        np.random.seed(4)
```

```
        self.mu = np.array(np.random.rand(self.n_components, self.features))
        #sigma 中为每个分模型都初始化了一个二维随机变量的协方差矩阵
        self.sigma = np.array([np.eye(self.features) / self.features] * self.n_components)
        self.alpha = np.array([1.0 / self.n_components] * self.n_components)
        print(self.alpha.shape, self.mu.shape, self.sigma.shape)
        print(self.alpha,self.mu,self.sigma)

    def gauss(self, Y, mu, sigma):   #调用多元正态分布的概率密度函数，计算高斯函数值
        return multivariate_normal.pdf(Y,mean=mu, cov=sigma )

    def preprocess(self, data):   #  数据预处理
        self.samples = data.shape[0]     #定义数据集大小
        self.features = data.shape[1]     #定义数据集的特征数
        pre = preprocessing.MinMaxScaler()   #进行了特征归一化
        return pre.fit_transform(data)

    def fit_predict(self, data):   #  拟合数据
        data = self.preprocess(data)     #进行数据预处理
        self._init(data)         #初始化模型参数
        weighted_probs = np.zeros((self.samples, self.n_components))
        print(weighted_probs.shape)   #用来存放进行 E 步后算出来的当前模型参数下每个观测数据
来自第 k 个分模型的概率  shape(150,3)
        for i in range(self.max_iter):
            prev_weighted_probs = weighted_probs
            #e 步
            weighted_probs = self._e_step(data)
            #change  当后验概率没有什么改变了的时候，即收敛的时候，停止迭代
            change = np.linalg.norm(weighted_probs - prev_weighted_probs)
            if change < self.error:
                break
            #m 步
            self._m_step(data, weighted_probs)
        #比较观测数据来自三个分模型的概率，返回概率最大的分模型所代表列的列号
        return weighted_probs.argmax(axis=1)

    def _e_step(self, data):   # E 步
        probs = np.zeros((self.samples, self.n_components))     #shape(150,3)
        for i in range(self.n_components):
            #调用类定义的 gauss 函数，计算不同高斯模型下数据集对应高斯函数值
            probs[:, i] = self.gauss(data, self.mu[i, :], self.sigma[i, :, :])

        weighted_probs = np.zeros(probs.shape)
        for i in range(self.n_components):
            weighted_probs[:, i] = self.alpha[i] * probs[:, i]
```

```
        for i in range(self.samples):
            #后验概率某类概率再除以三类的概率和
            weighted_probs[i, :] /= np.sum(weighted_probs[i, :])

        return weighted_probs

    def _m_step(self, data, weighted_probs):   # M 步   进行 mu,sigma,alpha 的数值更新
        for i in range(self.n_components):
            #求出每一列的概率和 即每行数据属于某一具体类的概率和
            sum_probs_i = np.sum(weighted_probs[:, i])
            #axis=0 计算每列的 sum
            self.mu[i, :] = np.sum(np.multiply(data, np.mat(weighted_probs[:, i]).T), axis=0) / sum_probs_i

            self.sigma[i, :, :] = (data - self.mu[i, :]).T * np.multiply((data - self.mu[i, :]), np.mat(weighted_probs[:, i]).T) / sum_probs_i

            #行数   shape[0]
            self.alpha[i] = sum_probs_i / data.shape[0]


iris_data=pd.read_csv("Iris/iris.data", header=None, names=['sepal length', 'sepal width', 'petal length', 'petal width', 'class'])

#发现有 iris-setosa    iris-versicolor    iris-virginica 三种
print(iris_data['class'].value_counts())
labels=iris_data['class'].values

#对 labels 进行标签编码
label_encoder=LabelEncoder()
labels=label_encoder.fit_transform(labels)
# print(labels)

#挑选 sepal length、petal length 两维特征作为聚类依据
x_axis=iris_data['sepal length']   #series(150,)
y_axis=iris_data['petal length']

data=np.array(pd.concat([x_axis,y_axis], axis=1))

gmm = GMM_EM(3)
pre_label = gmm.fit_predict(data)

print(pre_label)
print(labels)
```

```
#混合高斯算法求出的聚类中心
num_0, num_1, num_2=[0,0,0]
xsum_0, xsum_1, xsum_2=[0,0,0]
ysum_0, ysum_1, ysum_2=[0,0,0]

for i in range(len(pre_label)):
    if pre_label[i]==0:
        num_0+=1
        xsum_0+=x_axis[i]
        ysum_0 += y_axis[i]
    elif pre_label[i]==1:
        num_1+=1
        xsum_1+=x_axis[i]
        ysum_1 += y_axis[i]
    else:
        num_2+=1
        xsum_2+=x_axis[i]
        ysum_2 += y_axis[i]

x_center_0=xsum_0/num_0
y_center_0=ysum_0/num_0
x_center_1=xsum_1/num_1
y_center_1=ysum_1/num_1
x_center_2=xsum_2/num_2
y_center_2=ysum_2/num_2

x_center=[x_center_0, x_center_1, x_center_2]
y_center=[y_center_0, y_center_1, y_center_2]
#数据集的实际中心
x_ac_center=[x_axis[0:50].mean(), x_axis[50:100].mean(), x_axis[100:150].mean()]
y_ac_center=[y_axis[0:50].mean(), y_axis[50:100].mean(), y_axis[100:150].mean()]

#画图
#画混合高斯聚类图
plt.scatter(x_axis, y_axis, c=pre_label)
plt.scatter(x_center, y_center, c='r', marker='x')
plt.xlabel('sepal length')
plt.ylabel('petal length')
plt.show()

#实际数据图
plt.scatter(x_axis,y_axis,c=labels)
plt.scatter(x_ac_center,y_ac_center,c='r',marker='x')
```

```
plt.xlabel('sepal length')
plt.ylabel('petal length')
plt.show()

# EM 算法是对初始值敏感的，修改初始化的值会发现模型性能变化很大

#计算准确率
#分别计算三种类别组合   0 1 2      1 0 2     0 2 1      1 2 0      2 1 0     2 0 1
y_pred_1=np.array(np.zeros(150))
y_pred_2=np.array(np.zeros(150))
y_pred_3=np.array(np.zeros(150))
y_pred_4=np.array(np.zeros(150))
y_pred_5=np.array(np.zeros(150))
y_pred_6=np.array(np.zeros(150))
for i in range(150):
    if pre_label[i]==0:
        y_pred_1[i]=0
        y_pred_2[i] = 1
        y_pred_3[i] = 0
        y_pred_4[i] = 1
        y_pred_5[i] = 2
        y_pred_6[i] = 2
    if pre_label[i]==1:
        y_pred_1[i] = 1
        y_pred_2[i] = 0
        y_pred_3[i] = 2
        y_pred_4[i] = 2
        y_pred_5[i] = 1
        y_pred_6[i] = 0
    if pre_label[i]==2:
        y_pred_1[i] = 2
        y_pred_2[i] = 2
        y_pred_3[i] = 1
        y_pred_4[i] = 0
        y_pred_5[i] = 0
        y_pred_6[i] = 1

def correct_rate(lei_list):
    correct_num = 0
    for i in range(150):
        if (lei_list[i] == labels[i]):
            correct_num += 1
    rate = correct_num / 150
    return rate
```

```
rate1=correct_rate(y_pred_1)
rate2=correct_rate(y_pred_2)
rate3=correct_rate(y_pred_3)
rate4=correct_rate(y_pred_4)
rate5=correct_rate(y_pred_5)
rate6=correct_rate(y_pred_6)

#比较
rate=[rate1,rate2,rate3,rate4,rate5,rate6]
max_rate=0
for i in range(6):
    if rate[i]>max_rate:
        max_rate=rate[i]

print('准确率为：', max_rate)
```